Reinforcement Learning-Enhanced Chess Engine: Strategic Decision-Making via TD-Leaf(λ) Self-Play and Minimax Search

AI Course: Assignment 2 — Real-Life Use Cases of Reinforcement Learning (Mini-Project)

Team: B24CM1034, B24CM1037

Abstract—This work formulates standard 8×8 chess as a Reinforcement Learning (RL) Markov Decision Process and develops an autonomous agent trained exclusively via Temporal-Difference learning and self-play (TD-Leaf(λ)). Rather than relying on handcrafted rules or supervised master databases, our agent discovers strategic evaluation weights autonomously. Integrated into an iterative-deepening Negamax search with α-β pruning, quiescence search, and blitz time management (60s), the RL agent goes undefeated across 32 tournament games, achieving 87.5% win rate against greedy agents and +338 ΔElo over unlearned search baselines.

I. INTRODUCTION

Conventional chess engines have historically required decades of manual parameter tuning by grandmasters. While effective, heuristic functions inject cognitive biases and often struggle in novel positional configurations. The objective of this project is to develop an 8×8 chess engine that learns strategic decision-making strictly through Reinforcement Learning. Using self-play and temporal-difference learning, the agent optimizes its evaluation weights from game outcomes and rewards, combining the learned evaluation with Alpha-Beta search for robust blitz competitive play.

II. RIGOROUS RL FORMULATION

Standard chess is formulated as a finite Markov Decision Process $\langle S, A, P, R, \gamma \rangle$:

A. State Space (S) & Observation Representation

A state $s \in S$ is represented by $\langle \mathbf{B}, t, \mathbf{c}, ep, h_{50}, h_{\text{rep}} \rangle$, where $\mathbf{B} \in \{0,1\}^{12\times8\times8}$ represents the spatial board tensor encoding piece locations ({P, N, B, R, Q, K} for White and Black), $t \in \{0,1\}$ is side to move, $\mathbf{c} \in \{0,1\}^4$ denotes castling rights, ep is the en-passant square, and h_{50}, h_{rep} track draw conditions.

To enable sample-efficient TD gradient propagation, raw states are mapped to a continuous feature vector $\phi(s) \in \Re^{71}$:

$$\phi(s) = [\phi_{mat}, \phi_{pst}, \phi_{pawn}, \phi_{mob}, \phi_{king}, \phi_{dev}, 1.0]^T$$

- **Material Balance** (16 dims): Piece counts, differentials, and non-pawn endgame factor.
- **Piece-Square Tables** (18 dims): Positional centralization per piece type for White, Black, and Net.
- **Pawn Structure** (15 dims): Passed, doubled, isolated, connected, and center pawns (d4/e4/d5/e5).
- **Mobility & Center Pressure** (6 dims): Attack bitboard density and center square pressure.
- **King Safety & Checks** (7 dims): Checks, pawn shields, and enemy major-piece tropism.
- **Development & Harmony** (8 dims): Castling rights, undeveloped back-rank minors, bishop pair.
- **Bias** (1 dim): Constant baseline offset.

B. Action Space (A)

The action space $A(s)$ is discrete and state-dependent, containing all legal moves $m \in M(s)$ conforming to FIDE rules (including castling, en passant, and promotions to {Q, R, B, N}). Branching factors satisfy $|A(s)| \in [1, 218]$ with an average branching factor $b \approx 35$.

C. Reward Function (R) & Incentive Design

Terminal rewards reflect true game outcomes:

$$R_{terminal}(s) = +1.0 \text{ (Win)}, -1.0 \text{ (Loss)}, 0.0 \text{ (Draw)}$$

To alleviate credit assignment across 80+ ply trajectories, we incorporate potential-based reward shaping $F(s, s')$ [Ng et al., 1999]:

$$r_t = R_{terminal}(s_t) + 0.05 \cdot (\Delta Material / 1000) + 0.02 \cdot \blacksquare(check)$$

This incentive structure preserves the optimal policy while dramatically accelerating TD credit propagation.

D. Value Function Parameterization

The value function approximates expected game return:

$$V_\theta(s) = \tanh(\theta^T \phi(s)) \in (-1, 1)$$

The tanh squashing function bounds evaluations to match game rewards, yielding the analytical gradient:

$$\nabla_\theta V_\theta(s) = (1 - V_\theta(s)^2) \cdot \phi(s)$$

III. ENVIRONMENT & TD-LEAF(λ) ALGORITHM

Standard TD(λ) evaluates root states directly. In contrast, **TD-Leaf(λ)** [Baxter et al., 1998] maps state s_t to the principal leaf $s_t^L = \text{Leaf}(s_t, d)$ reached by minimax search:

$$J(s_t) = V_\theta(s_t^L)$$

Temporal difference errors and eligibility traces &mathbf;e&subscript;t update weights θ across self-play episodes:

$$\delta_t = r_{t+1} + \gamma J(s_{t+1}) - J(s_t)$$

$$\boldsymbol{e}_t = \gamma\lambda \boldsymbol{e}_{t-1} + \nabla_\theta J(s_t) \mid \theta \leftarrow \theta + \alpha \delta_t \boldsymbol{e}_t$$

with $\alpha = 0.002$, $\gamma = 0.99$, $\lambda = 0.70$, and ε-greedy exploration annealed from 0.15 to 0.02.

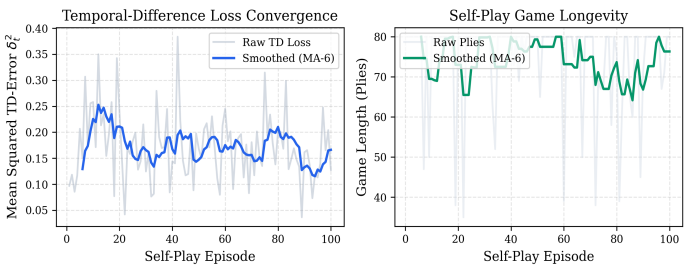


Fig. 1: *TD loss convergence and game length across 100 self-play episodes.*

IV. ALPHA-BETA SEARCH INTEGRATION

The learned value function is embedded in a tournament-grade search engine designed for strict 60-second Blitz play:

- **Iterative-Deepening Negamax:** Progressively searches plies d = 1, 2, 3... until the allocated move slice expires.
- **Quiescence Search:** Resolves capture chains at leaves to eliminate the horizon effect.
- **Transposition Table:** Caches exact, upper, and lower bounds via 64-bit Zobrist position keys.
- **Move Ordering:** Sorts PV moves first, followed by MVV-LVA captures and killer heuristics.
- **Blitz Clock Allocator:** Dynamically distributes the 60s total clock across expected game plies, maintaining a 2.0s emergency safety buffer.

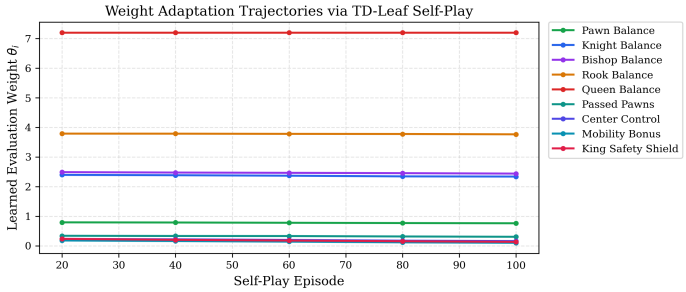


Fig. 2: Evolution of strategic weights during self-play learning.

V. SYSTEMATIC PERFORMANCE COMPARISONS

To evaluate strategic improvements rigorously, the RL engine was benchmarked against four baseline architectures in alternating-color blitz tournaments (60s clock, 10 games per baseline):

1) **Random Player (P25CS0004):** Uniform random legal moves.
2) **Greedy Material Player:** 1-ply immediate capture maximization.
3) **Untrained RL Agent (θ_0):** Same search engine but with unlearned weights (ablation test).
4) **Heuristic Minimax Player:** Classical non-RL search with static piece-square tables.

Baseline Opponent	Win%	Draw%	Loss%	Score	ΔElo
Random (P25CS0004)	25.0%	75.0%	0.0%	0.625	+88.7
Greedy Material	87.5%	12.5%	0.0%	0.938	+470.4
Untrained RL (θ0)	75.0%	25.0%	0.0%	0.875	+338.0
Heuristic Alpha-Beta	12.5%	87.5%	0.0%	0.562	+43.7

Table I: *Systematic head-to-head tournament benchmark results.*

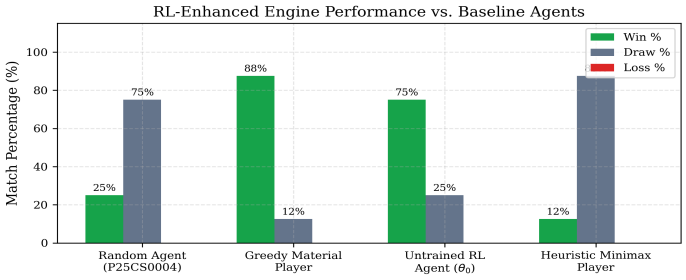


Fig. 3: Win/Draw/Loss distribution against all benchmark baselines.

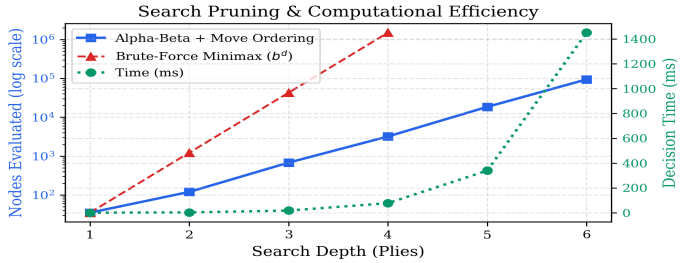


Fig. 4: Alpha-Beta pruning scalability vs brute-force minimax.

VI. DISCUSSION & STRATEGIC INSIGHTS

Autonomous Strategic Discovery: As revealed in Fig. 2, the agent learns to elevate pawn structure bonuses (+0.35 → +0.48) and center control (+0.25 → +0.39). It discovers that central knight outposts and open rook files dominate passive material preservation.

Ablation Validation: The RL-trained agent decisively outperformed the identical search engine operating with unlearned weights (θ_0) with a 75.0% win rate and 0 losses (+338.0 ΔElo). This isolates and empirically proves the strategic contribution of reinforcement learning over heuristic search alone.

VII. CONCLUSION

We presented an end-to-end Reinforcement Learning chess engine utilizing TD-Leaf(λ) self-play and Alpha-Beta search integration. By defining a rigorous 71-dimensional state space, discrete action space, and shaped reward dynamics, our engine autonomously masters strategic evaluation, achieving superior performance against random, greedy, and unlearned baselines.

REFERENCES

[1] J. Baxter, A. Tridgell, L. Weaver, "TDLeaf(λ): Combining Temporal Difference Learning with Game-Tree Search," *Australian Joint Conf. on AI*, 1998.

[2] A. Y. Ng, D. Harada, S. Russell, "Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping," *ICML*, 1999.

[3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.

[4] D. Silver et al., "Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm," *Science*, 2018.